Task 3 : Encapsulation Demo

```
/*
Name : Pravin Kumar Thakur
Roll no : 04
Task No : 03
Topic : Encapsulation Demo
Date : 04-03-2026
*/
Class Employee
 private int empId
 private string name
 private double salary;
 public int getEmpId() { return empID; }
public void setEmpId (int empId) { this.empID = empId; }

public string getName() { return name; }
public string getName() { return name; }
public double getSalary (String name) { this.name; }
                                        { return salary; }

public void setSalary (double S){
 if (S>0) this.salary = S;
 else System.out.println("Invalid salary!");
 }

}
```

```
public class Task3 {
public static void main(String[] args){
    Employee emp = new Employee();

    emp.setEmpId(101);
    emp.setName("Manish");
    emp.setSalary(50000);
System.out.println("ID"+emp.getEmpId()+", Name:"
    + emp.getName()+", Salary:Rs."+emp.getSalary());
    }

}
```

Task 3 Sorting - Bubble Sort & Selection Sort

```
#include <stdio.h>

void printArray (int a[], int n) {
  for (int i=0; i<n; i++) printf ("%d ", a[i]);
  printf ("\n")
}
int main()
  int n, bswaps = 0, sswaps = 0
  printf ("Enter the no of elements: ");
  scanf ("%d", &n);
  int bArr[n], sArr[n];

  for (int i=0 i<n; i++) {
    scanf ("%d", &bArr[i]);
    sArr[i] = bArr[i];
  printf ("\nOriginal Array: ");
  printArray (bArr, n);

printf("\n --- Bubble sort Pass-by Pass --- \n");
  for(int i=0; i<n-1; i++){
    for (int j=0; j<n-i-1; j++){
```

```
if (bArr[i] > bArr[j+1]){
    int t = bArr[j]; bArr[j] = bArr[j+1]; bArr[j+1] = t;
    bSwaps++;
  }
 }
 printf("Pass %d: ", i+1);
 printArray(bArr, n);
}

for (int i=0; i<n-1; i++){
 int min = i;
 for (int j=i+1; j<n; j++){
    if (sArr[j] < sArr[min]) min = j;
  }

 if (min != i){
  int t = sArr[i]; sArr[i] = sArr[min]; sArr[min] = t;
    sSwaps++;
 }
}
printf("\nFinal sorted Arrays: ");
printArray(sArr, n);
printf("\n\n-comparision of swaps ...\n");
printf("Bubble sort swaps: %d\nSelection sort: %d\n", bSwaps, sSwaps);

return 0;
}
```

S.No.	Form of the rational function	Form of the partial fraction
1.	$\frac{px+q}{(x-a)(x-b)}, a \neq b$	$\frac{A}{x-a}+\frac{B}{x-b}$
2.	$\frac{px+q}{(x-a)^2}$	$\frac{A}{x-a}+\frac{B}{(x-a)^2}$
3.	$\frac{px^2+qx+r}{(x-a)(x-b)(x-c)}$	$\frac{A}{x-a}+\frac{B}{x-b}+\frac{C}{x-c}$
4.	$\frac{px^2+qx+r}{(x-a)^2(x-b)}$	$\frac{A}{x-a}+\frac{B}{(x-a)^2}+\frac{C}{x-b}$
5.	$\frac{px^2+qx+r}{(x-a)(x^2+bx+c)}$ where $x^2 + bx + c$ cannot be factorised further	$\frac{A}{x-a}+\frac{Bx+C}{x^2+bx+c}$,

4 Singly Linked List - Create, Insert & Delete

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node* next; };

struct Node* createNode (int val){
    struct Node* n = (struct Node*) malloc (sizeof(struct Node));

    n-> data = val; n-> next = NULL;
    return n;
}
void display (struct Node* head){
    while(head) { printf("%d -> ", head -> data); head = head->next;}
    printf ("NULL\n");

}
void insertAtBeginning(struct Node** head, int Val){
    struct Node* n = createNode (val);
    n -> next = *head; *head = n;

}
void insertAtEnd (struct Node** head, int val){
    struct Node* n = createNode (val);
    if (!*head) *n = createNode (val);
    struct Node* t = *head;
    while (t -> next) t = t -> next;
    t -> next = n;
```